

Apuntes de PR3 Versión básica

Creación RDS con MySQL para DataBase

1. Acceder a Aurora RDS y darle a crear una base de datos.
2. Opciones del motor: MariaDB.
3. Plantilla: entorno de pruebas.
4. Configuración de la base de datos:
 - Identificador de la instancia: nombre de la base de datos.
 - Usuario maestro: admin.
 - Administración de credenciales: autoadministrado.
 - Contraseña maestra: 12345678.
5. Conectividad--> Acceso público: Sí.
6. Conectividad--> Grupo de seguridad de VPC: Crear nuevo grupo de seguridad. (si solo aparece default, configurar uno nuevo EC2-->Seguridad-->Grupos de seguridad-->Crear grupo de seguridad).
7. Configuración adicional--> Habilitar copia de seguridad: deshabilitado.
8. onfiguración adicional--> Habilitar el cifrado: deshabilitado.
9. Crear base de datos.

Crear Base de Datos MySQL en RDS

1. Abrir terminal WSL.
2. Instalar cliente MySQL: `sudo apt-get install mysql-client`.
3. Copiar el endpoint del RDS. Desde Aurora and RDS de AWS: base de datos-->Conectividad y seguridad-->Endpoint.
4. Conectarse a la base de datos:

```
$mysql -h <endpoint_copiado> -u <usuario_maestro punto 4> -p
```

5. Una vez dentro de MariaDB, crear la base de datos:

```
CREATE DATABASE <nombre_base_de_datos>;
USE <nombre_base_de_datos>;
CREATE TABLE <nombre_tabla> (
  user VARCHAR(255),
  attachment VARCHAR(255),
  comment VARCHAR(255)
);
```

Configuración de S3 buckets para almacenamiento de archivos

1. Acceder a S3 y darle a *crear bucket*.

2. Nombre del bucket: nombre único.
3. Propiedad del objeto--> ACL habilitadas.
4. Configuración de bloqueo de acceso público: DESMARCAR *Bloquear todo el acceso público*.
5. Cifrado predeterminado: Clave de bucket--> MARCAR *Desactivar*.
6. Crear bucket.

Configuración de permisos del bucket S3

1. Acceder al bucket creado (Amazon S3 > Buckets > nombre_bucket).
2. Ir a la pestaña de permisos.
3. Editar la política del bucket (Política de bucket --> Editar).
4. Copiar y pegar el archivo:

```
{

  "Version": "2012-10-17",

  "Statement": [
    {
      "Sid": "AddPerm",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::NOMBREDEBUCKETCREADO/*"
    }
  ]
}
```

5. Cambiar NOMBREDEBUCKETCREADO por el nombre del bucket creado.
6. Guardar cambios.

Configuración de Lambda

Creación de función Lambda estándar

1. Acceder a Lambda y darle a crear función.
2. Crear desde cero.
3. Información básica:
 - Nombre de la función: <nombre_función>.
 - Versión ejecutable: Python 3.14.
 - Cambiar rol de ejecución predeterminado--> MARCAR *Uso de un rol existente*.
 - Rol existente: *LabRole*.
4. Configuraciones adicionales:

- Redes:
 - MARCAR *Activar URL de función*.
 - Tipo de autorización--> MARCAR *None*.

5. Crear función.

Subida de archivos a S3

1. Desde (Lambda > Funciones > <nombre_función>), vamos a código fuente.
2. Modificamos el código fuente por:

```
import json

import sys, os, base64, datetime, hashlib, hmac

bucket = "NOMBRE_DEL_BUCKET_DONDE_SE_CONECTARÁ"
bucketUrl = "https://NOMBRE_DEL_BUCKET.s3.us-east-1.amazonaws.com/"
region = 'us-east-1'
service = 's3'

t = datetime.datetime.utcnow()
amzDate = t.strftime('%Y%m%dT%H%M%SZ')
dateStamp = t.strftime('%Y%m%d') # Date w/o time, used in credential scope

# Key derivation functions. See:
# http://docs.aws.amazon.com/general/latest/gr/signature-v4-examples.html#signature-v4-examples-python
def sign(key, msg):
    return hmac.new(key, msg.encode('utf-8'), hashlib.sha256).digest()

def getSignatureKey(key, dateStamp, regionName, serviceName):
    kDate = sign(('AWS4' + key).encode('utf-8'), dateStamp)
    kRegion = sign(kDate, regionName)
    kService = sign(kRegion, serviceName)
    kSigning = sign(kService, 'aws4_request')
    return kSigning

def lambda_handler(event, context):
    # TODO implement
    #aws_access_key_id=event["queryStringParameters"]["ak"];
    #aws_secret_access_key=event["queryStringParameters"]["sk"];
    #aws_session_token=event["queryStringParameters"]["st"];

    ``````

```

> A continuación, para las variables AWS, hay que ir a la siguiente página, clicar en AWS Details y luego en Show para copiar el contenido

seleccionado, obteniendo un resultado similar al `del` código. IMPORTANTE: se necesita copiar y pegar cada vez que se lance el laboratorio.

```
> ![aws_vars_config](./aws_vars_config.png)
```

```

```

```
```python
```

```
    aws_access_key_id="ASIA47CRXX0PUZF3VSYH"
```

```
    aws_secret_access_key="SpqcYstYU0+VFFsSRFACyBK3o+xbBVRqqTJpJsZ+"
```

```
aws_session_token="IQoJb3JpZ2luX2VjEHIAcXVzLXdlc3QtMiJHMEUCIQCDV65TuT2KC6qMM
3VTZonXraXJTUPfs9m0SRbWSRyTaQIgPbYhQT9+qjoFic1XNAK1dmOGhdOHbCnUVPTYMXMzaeQqs
wII0xAAGgw40TEzNzcXmZg1OTEiDDEbtGjfgfT2cDgi/CqQAiu+2aw39Fj8t7Ta1UYUd8tA66FUn
fMVm+eQEm7KRjNu4UxqbRYUkJ4Q5b27DdGk7C08Py20P7NXahi47rcLBZLnHsccY/rGLoL4XoASS
x9EIjrJ00XWStc2utIfdgwSGAv0tvYaNTTYoUgNP6eb6HpP2z96bXCbBNXgM9mQN1AV85X6hp4H/
xJ7yTtX71i3nw157/8PwExAqAFPdiD/zUh5XyC4g4Mz2qmbaS2A0EcgcwmwggA8aILUZWcufkDC7
liHpcYcZZJJVrFBEhNVQQ4pPYP3z22CR7Qj3itqGRcI9cXJ7cF7c0a+qyMP0oojwBlarcnCCSujk
B0XPjxCARkv8UFPou8F69xQFuIOVQ7iMPHn+8kG0p0B3ycELL+7cCWAmnlio2wSOMYL0Bj8Kdgo0
aioswh8faGcY0joHyklmfCc60lqurDh13DFFNgQSV1v+JMkKw0ZihLtOPH4EliFCDzVgv8PIVxke
LvxD02u9tIOVwNojMkBjvkNe0ZvjihQwAv5VxE3vvZk15shd3G/3jSh1/2gXgRdtp0dXvkCFUKY5
vpfRx/ecBIIYhxfRp7TaxvEeA=="
```

```
    stringToSign= b""
```

```
    policy = """{"expiration": "2026-12-30T12:00:00.000Z",
"conditions": [
{"bucket": \"\"\" + bucket + \"\"\"\"},
["starts-with", "$key", ""],
{"acl": "public-read"},
{"success_action_redirect": \"\"\"+ bucketUrl+\"\"\"success.html"},
{"x-amz-credential": \"\"\"+
aws_access_key_id+"/"+dateStamp+"/"+region+\"\"\"/s3/aws4_request"},
{"x-amz-algorithm": "AWS4-HMAC-SHA256"},
{"x-amz-date": \"\"\"+amzDate+\"\"\"\" },
{"x-amz-security-token": \"\"\" + aws_session_token + \"\"\"\" }
]
}"""
```

```
    stringToSign=base64.b64encode(bytes((policy).encode("utf-8")))
```

```
    signing_key = getSignatureKey(aws_secret_access_key, dateStamp, region,
service)
```

```
    signature = hmac.new(signing_key, stringToSign,
hashlib.sha256).hexdigest()
```

```
    #print(dateStamp)
```

```
    #print(signature)
```

```
    print(policy)
```

```
    return {
```

```
        'statusCode': 200,
```

```
        'headers': { 'Access-Control-Allow-Origin' : '*' },
```

```
'body':json.dumps({ 'stringSigned' : signature , 'stringToSign' :
stringToSign.decode('utf-8') , 'xAmzCredential' :
aws_access_key_id+"/"+dateStamp+"/"+region+ "/s3/aws4_request" , 'dateStamp'
: dateStamp , 'amzDate' : amzDate , 'securityToken' : aws_session_token })
}
```

3. Deploy changes.

Twitter post, añadir Posts a BBDD

1. Desde (Lambda > Funciones > <nombre_función>), vamos a código fuente.
2. Subimos el zip suministrado cómo código fuente (Cargar desde): *twitterPost.zip*.
3. Veremos algo cómo en código fuente:

```
import sys
import logging
import pymysql
import json
import base64
from urllib.parse import parse_qs

# MODIFICAR
rds_host = "ENDOPOYNT_DEL_RDS"

username = "USUARIO_MAESTRO_MARIA_DB"
password = "CONTRASEÑA_MARIA_DB"
dbname = "NOMBRE_DE_LA_BASE_DE_DATOS"

bucketUrl = "https://NOMBRE_DEL_BUCKET.s3.us-east-1.amazonaws.com/"

def lambda_handler(event , context):
    print(json.dumps(event));

    user="err";
    comment="err";
    attachment="err";
    user=event["queryStringParameters"]["user"];
    comment=event["queryStringParameters"]["comment"];
    attachment=bucketUrl+event["queryStringParameters"]["attachment"];

    print(user);

    try:
        conn = pymysql.connect(rds_host, user=username, passwd=password,
db=dbname, connect_timeout=10, port=3306)
        print(1);
        with conn.cursor() as cur:
            cur.execute("insert into posts
values('"+user+"','"+comment+"','"+attachment+"')");
            print(2);
```

```

        conn.commit();
        print(3);
        cur.close();
    except pymysql.MySQLError as e:
        print(e)
    conn.close();
    print(4);
    return {
        'statusCode': 200,
        'headers': { 'Access-Control-Allow-Origin' : '*' },
        'body' : 'OK'
    }

```

4. Deploy changes.

Twitter read, leer Posts de la BBDD

1. Desde (Lambda > Funciones > <nombre_función>), vamos a código fuente.
2. Subimos el zip suministrado como código fuente (Cargar desde): *twitterPost.zip*.
3. Veremos algo como en código fuente:

```

import sys
import logging
import pymysql
import json
import base64
from urllib.parse import parse_qs

# MODIFICAR
rds_host = "ENDOPOYNT_DEL_RDS"

username = "USUARIO_MAESTRO_MARIA_DB"
password = "CONTRASEÑA_MARIA_DB"
dbname = "NOMBRE_DE_LA_BASE_DE_DATOS"

bucketUrl = "https://NOMBRE_DEL_BUCKET.s3.us-east-1.amazonaws.com/"

def lambda_handler(event, context):
    print(json.dumps(event));

    user="err";

    print(user);

    commentList="{\"posts\": [";
    try:
        conn = pymysql.connect(rds_host, user=username, passwd=password,
                               db=dbname, connect_timeout=10, port=3306)
        print(1);
        with conn.cursor() as cur:

```

```

        #cur.execute("select * from posts where username='"+user+"'");
        cur.execute("select * from posts");
        print(2);
        rows = cur.fetchall()
        print(3);
        for row in rows:
            commentList+="{\"user\": \""+row[0]+"\", \"comment\": \"+row[1].replace("\n", "\\n")+"\", \"attachment\": \""+row[2]+"\"},";

        commentList=commentList[:-1]+"]}";
        cur.close();
        print(commentList);
    except pymysql.MySQLError as e:
        print (e)
    conn.close();
    print(4);
    return {
        'statusCode': 200,
        'headers': { 'Access-Control-Allow-Origin' : '*' },
        'body' : commentList    }

```

4. Deploy Changes.

Objetos de interfaz para el bucket

Archivos HTML

- HTML postComment

```

<!DOCTYPE html>
<html>
  <head>
    <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>
    <script>
      const syncWait = (ms) => {
        const end = Date.now() + ms;
        while (Date.now() < end) continue;
      };

      function getAWSKeys() {
        var asd = $.get(
          //
          // SUSTITUIR POR LA URI DE LA FUNCIÓN LAMBDA CONNECT
          // (Lambda > Funciones > <nombre_de_la_función> > información general)
          //
          "https://zuemvmxfafji6udipjgrmiumnm0hsquy.lambda-url.us-east-1.on.aws/",
          {},
          function (data) {
            var json = data;

```

```

        //      json=JSON.parse(json);
        document.getElementById("Policy").value = json.stringToSign;
        document.getElementById("X-Amz-Credential").value =
            json.xAmzCredential;
        document.getElementById("X-Amz-Date").value = json.amzDate;
        document.getElementById("X-Amz-Signature").value =
            json.stringSigned;
        document.getElementById("X-Amz-Security-Token").value =
            json.securityToken;
    }
    );
}

function setKeyFilename() {
    alert("Enviando!");

    document.getElementById("key").value = document
        .getElementById("file")
        .value.substring(
            document.getElementById("file").value.lastIndexOf("\\") + 1
        );

    var post = document.getElementById("post").value;
    var file = document.getElementById("key").value;
    var user = document.getElementById("username").value;
    //
    // SUSTITUIR POR LA URI DE LA FUNCIÓN LAMBDA POST
    // (Lambda > Funciones > <nombre_de_la_función> > información general)
    //
    var sendPost = $.get(
        "https://czsboxcschlaqfzgykh4vwmgile0vivmg.lambda-url.us-east-1.on.aws/",

        { user: user, comment: post, attachment: file },
        function (data) {
            function jsonEscape(str) {
                return str
                    .replace(/\n/g, "\\n")
                    .replace(/\r/g, "\\r")
                    .replace(/\t/g, "\\t")
                    .replace(/\\/g, "");
            }

            var json = data;
            alert("resultado " + JSON.stringify(json));
        }
    );
    /*
        .done(function() {
            alert( "second success" );
        })
        .fail(function(xhr, status, error) {
            alert( error );
        })

        sendPost.always(function() {

```



```

        alert( "second finished" );
    });
    */
    syncWait(3000);
}
</script>
</head>
<body onload="getAWSKeys()">
    <label for="username">Nombre de usuario:</label>
    <input
        type="text"
        id="username"
        name="username"
        value="userPrueba"
    /><br /><br />
    <label for="post">Escribe un mensaje:</label>
    <textarea id="post" name="post" rows="5" cols="33"> </textarea>

    <!--> MODIFICAR URI PARA ACCEDER AL BUCKET <!-->
    <form
        action="https://NOMBRE_DEL_BUCKET.s3.us-east-1.amazonaws.com"
        onsubmit="setKeyFilename()"
        method="post"
        enctype="multipart/form-data"
    >
        <input
            type="hidden"
            id="X-Amz-Credential"
            name="X-Amz-Credential"
            value=""
        />
        <input type="hidden" id="X-Amz-Date" name="X-Amz-Date" value="" />
        <input type="hidden" id="Policy" name="Policy" value="" />
        <input
            type="hidden"
            id="X-Amz-Signature"
            name="X-Amz-Signature"
            value=""
        />
        <input type="hidden" id="key" name="key" value="fichero.sln" /><br />
        <input type="hidden" name="acl" value="public-read" />
        <input
            type="hidden"
            name="success_action_redirect"
            value="https://NOMBRE_DEL_BUCKET.s3.us-east-1.amazonaws.com/success.html"
        />
        <input type="hidden" name="X-Amz-Algorithm" value="AWS4-HMAC-SHA256" />
        <input
            type="hidden"
            id="X-Amz-Security-Token"
            name="X-Amz-Security-Token"
            value=""
        />

```

```

<label for="file">Adjuntar un fichero:</label>
<input
  type="file"
  name="file"
  id="file"
  accept="video/mp4,image/png"
/><br /><br />
<input type="submit" value="Subir fichero" name="submit" />
</form>
</body>
</html>

```

- HTML readPosts

```

<!DOCTYPE html>
<html>
  <head>
    <script src="https://code.jquery.com/jquery-1.9.1.min.js"></script>
    <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.1/jquery.min.js"></script>

    <script>
      $(document).ready(function () {
        var user = "userPrueba";
        var sendPost = $.get(
          //
          // SUSTITUIR POR LA URI DE LA FUNCIÓN LAMBDA
          // (Lambda > Funciones > <nombre_de_la_función> > información general)
          //
          "https://aae7bk7xxohasox2ua6dxe7uyq0bukob.lambda-url.us-east-1.on.aws/",

          { user: user },
          function (data) {
            // function jsonEscape(str) {
            //   return str.replace(/\n/g, "\\n").replace(/\r/g,
            "\\r").replace(/\t/g, "\\t").replace(/\\/g, "");
            // }
            var json = data;
            alert("resultado " + JSON.stringify(json));
            for (let i in json.posts) {
              var ul = document.getElementById("comment_list");
              var li = document.createElement("li");
              var comment = document.createElement("p");
              comment.innerText =
                json.posts[i].user + " dice:" + json.posts[i].comment;
              var video = document.createElement("video");
              video.setAttribute("width", "320");
              video.setAttribute("height", "320");
              video.setAttribute("controls", "");

              var source = document.createElement("source");
              source.setAttribute("src", json.posts[i].attachment);
            }
          }
        );
      });
    </script>
  </head>
  <body>
    <div id="comment_list">
    </div>
  </body>
</html>

```

```

        source.setAttribute("type", "video/mp4");
        video.appendChild(source);
        ul.appendChild(li);
        li.appendChild(comment);
        li.appendChild(video);
    }
}
);
});
</script>
</head>
<body>
<ul id="comment_list">
<!-- <li>
    RESULTADO ESPERADO DEL SCRIPT:

    <p> Comentario 1</p>
    <video width="320" height="240" controls>
    <source src="https://utadbucket1.s3.us-east-1.amazonaws.com/dp1.mp4"
type="video/mp4">

    Your browser does not support the video tag.
    </video>
</li>
<li>
    <p> Comentario 2</p>
    <video width="320" height="240" controls>
    <source src="https://utadbucket1.s3.us-east-1.amazonaws.com/dp1.mp4"
type="video/mp4">

    Your browser does not support the video tag.
    </video>
</li>
<li>
    <p> Comentario 3</p>
    <video width="320" height="240" controls>
    <source src="https://utadbucket1.s3.us-east-1.amazonaws.com/dp1.mp4"
type="video/mp4">

    Your browser does not support the video tag.
    </video>
</li>
--></ul>
</body>
</html>

```

Instrucciones para subir a los buckets

1. Acceder a S3 desde AWS.
2. Seleccionar Buckets--><Nombre_del_bucket>.
3. Pulsar *Cargar* y subir los archivos html.

TESTING

1. Copias bien todos lo códigos porque Python es un lenguaje de programación Beta.
2. Abres las páginas de *postComment.html* y *readPosts.html*.
3. Desde *postComment.html* tratas de subir un archivo y un texto.
4. Comprobar que aparece en *readPosts.html* y cómo objeto del bucket en el S3.
5. Si algo falla rezar, después abrir ClockWatch (Lambda > <nombre_de_la_función> > Mantenimiento > Abrir ClockWatch).

Apuntes de PR3 Versión avanzada

Creación de una nueva tabla en la base de datos para guardar contraseña y usuario

```
USE twitter;
CREATE TABLE users (
    user_name VARCHAR(255),
    password VARCHAR(255)
);
INSERT INTO users (user_name, password) VALUES ("Diego", "1234");
```

Creación de una función Lambda de login (comprobar usuario y contraseña)

```
import json
import pymysql

rds_host = "pr3-rds-twitter.cbeymmysczg.us-east-1.rds.amazonaws.com"
username = "admin"
password = "12345678"
dbname = "twitter"
dbport = 3306

headers = {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "GET,OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type",
    "Content-Type": "text/plain"
}

def lambda_handler(event, context):
    print(json.dumps(event))

    qs = (event.get("queryStringParameters") or {})
```

```
user = qs.get("user", "")
passwd = qs.get("password", "")

if not user or not passwd:
    return {
        "statusCode": 200,
        "headers": headers,
        "body": "KO"
    }

try:
    conn = pymysql.connect(
        host=rds_host, user=username, passwd=password,
        db=dbname, port=dbport, connect_timeout=10
    )
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM users WHERE users=%s AND password=%s LIMIT
1", (user, passwd))
        row = cur.fetchone()

    conn.close()

    return {
        "statusCode": 200,
        "headers": headers,
        "body": "OK" if row else "KO"
    }

except Exception as e:
    print(e)
    return {
        "statusCode": 200,
        "headers": headers,
        "body": "KO"
    }
```

Cargar zip twitterPost.zip, para obtener las librerías cargadas en su entorno virtual No poner los mismos nombres en todo

HTML con inputs copiado del postComment.html y adaptado

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Inicio de sesión</title>

    <script
src="https://ajax.googleapis.com/ajax/libs/jquery/3.4.1/jquery.min.js"></script>

    <script>
```

```
function setKeyFilename(event) {
    event.preventDefault();

    alert("Enviando!");

    const user = document.getElementById("username").value;
    const password = document.getElementById("password").value;

    $.get({
        url: "https://g2twjbxv4t3mebl3nlmbwewpe0higzj.lambda-url.us-east-1.on.aws/",
        method: "GET",
        data: { user, password },
        dataType: "text", // esto evita que intente parsear JSON
        cache: false,
        success: function (data) {
            if ((data || "").trim() === "OK") {
                localStorage.setItem("loggedUser", user);
                window.location.href = "postComment.html";
            } else {
                $("#error").text("Usuario o contraseña incorrectos.");
            }
        },
        error: function () {
            $("#error").text("Error llamando al servicio de login.");
        }
    });
}
</script>
</head>

<body>

    <label for="username">Nombre de usuario:</label>
    <input type="text" id="username" name="username"><br><br>

    <label for="password">Escribe un mensaje:</label>
    <input type="password" id="password" name="password"><br><br>

    <form onsubmit="setKeyFilename(event)">
        <input type="submit" value="Iniciar sesión">
    </form>

    <p id="error"></p>

</body>
</html>
```

Si me tengo que enterar de que es \$ajax en vez de \$get estamos jodidos.